TEAM P29 / PROJECT CASE STUDY

# eAUD

## Designing a permissioned central ba digital currency research prototype

A plain-English and technical account of the problem, system design, product, compliance model, engineering process and project outcomes.

**Academic research prototype**

Hyperledger Fabric / React / Node.js / Keycloak / Docker

**Project by Team P29**

Isar Ujoodah · Luvish Rajnath · Lasith Perera
Farzana Moietry · Ashikur Rahman · Al Hamid Arath

**Prepared by Isar Ujoodah**

Lead Developer & Scrum Master · Case-study author

Swinburne University of Technology · 2025-2026

PUBLIC PORTFOLIO EDITION / SEPTEMBER 2026

PUBLICATION NOTE

# About this case study

This document is a project case study, not a product brochure and not an academic submission. It explains the eAUD prototype in enough context for readers from software, finance, risk, compliance and general business backgrounds.

The eAUD project was completed by Team P29 at Swinburne University of Technology across 2025-2026. The project combined research, stakeholder and regulatory analysis with a working permissioned-blockchain prototype. The prototype modelled a central-bank role, two commercial-bank roles, a regulatory observer role and a customer role, supported by a React application, a Node.js API, Keycloak identity management and Hyperledger Fabric.

This public edition is prepared and authored by Isar Ujoodah. I served as Scrum Master and lead developer during the project, but the report intentionally documents the project as a team outcome. A dedicated section near the end describes team contributions and identifies the work I personally led or implemented.

**Important scope statement**

The system is an academic research prototype. It is not affiliated with, endorsed by, or operated by the Reserve Bank of Australia (RBA), AUSTRAC, APRA or any Australian bank. It does not issue legal tender and should not be interpreted as a production-ready financial system.

## How to read the report

Readers who are new to central bank digital currency can begin with the primer and Australian context. Finance and risk readers may then move to the stakeholder, transaction, AML/CTF and regulatory-design sections. Technical readers can continue into the architecture, identity, blockchain and deployment chapters. The product walkthrough uses screenshots from the working prototype and the final sprint evidence.

CONTENTS

# Report structure

01 / EXECUTIVE SUMMARY

# Executive summary

A team-built research prototype for exploring how tokenised central-bank money, institutional permissions, KYC and transaction monitoring might coexist in one controlled architecture.

## The project in one page

Team P29 investigated the design of an Australian eAUD concept through two connected activities: a case-study and regulatory/design stream, and a working software prototype. The initial brief called for architectural design, policy analysis, evaluation of open-source blockchain technology and an operational roadmap. The team then progressively implemented a prototype to make those design questions tangible.

The final system modelled four organisations on a permissioned Hyperledger Fabric network: an RBA role, Bank A, Bank B and an AUSTRAC observer role. A React frontend exposed role-specific workflows, a Node.js/Express API coordinated application logic, Keycloak provided centralised identity and role mapping, and custom JavaScript chaincode implemented wallet creation, eAUD funding and transfers. The application also included KYC document workflows, a seven-signal AML monitoring prototype, transaction history, email alerts, analytics, pagination/search, CSV/PDF export work and Docker-based deployment.

The most useful outcome was not a claim that blockchain “replaces” Australia’s current payment system. Australia already operates high-value real-time gross settlement through RITS, with final and irrevocable settlement in central-bank accounts. The case-study question is narrower and more contemporary: if wholesale financial markets become more tokenised, what might a permissioned digital-money platform need to coordinate identity, institutional roles, transaction logic, compliance monitoring and operational controls? This aligns more closely with the direction of the RBA and DFCRC’s Project Acacia, which explored digital money and settlement infrastructure for wholesale tokenised asset markets.

### What the prototype demonstrated

A working system can make governance questions concrete. Who is allowed to mint? Which institution can view which wallets? Where should KYC data live? When should suspicious-activity checks run? What should a regulator be able to observe without becoming a transaction participant? Those questions became system behaviour instead of remaining only in a report.

## At a glance

| Dimension | Prototype |
|---|---|
| Project | P29 - Designing Australia’s Central Bank Digital Currency |
| Period | 2025-2026, two academic project phases |

| Dimension | Prototype |
|---|---|
| | |
| Team | 6 members |
| Network model | 4 organisations: RBA, Bank A, Bank B, AUSTRAC |
| Application roles | RBA Admin, Bank A Admin, Bank B Admin, AUSTRAC, Customer |
| Blockchain | Hyperledger Fabric 2.5 / JavaScript chaincode / CouchDB |
| Application | React + Node.js / Express |
| Identity | Keycloak, OAuth 2.0 / OpenID Connect |
| Compliance features | KYC workflow, 7-signal AML risk engine, monitoring dashboard, watchlist and alerts |
| Deployment | Docker / Docker Compose for application services |
| Final validation | Client walkthrough and live demo on 3 June 2026 |

02 / CBDC IN PLAIN ENGLISH

# CBDC in plain English

A finance reader should not need blockchain experience, and a software reader should not need payments-market experience, to understand the rest of the report.

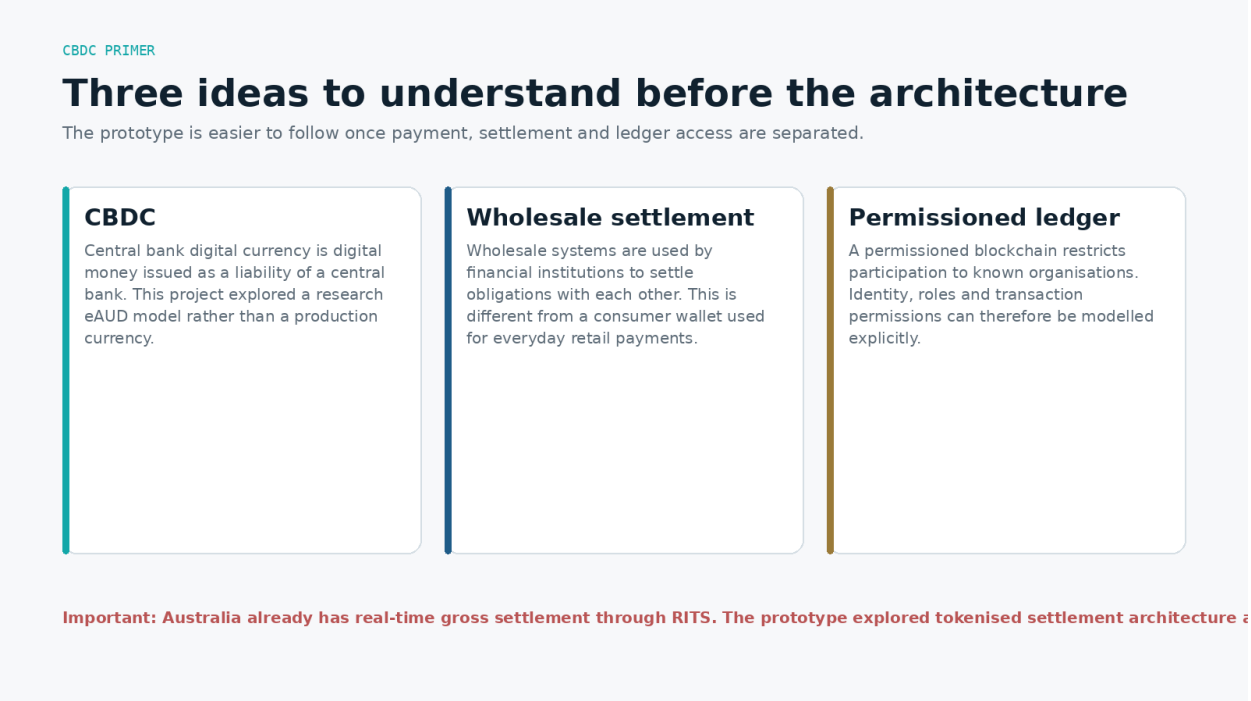



*Figure 1. Three concepts used throughout the case study.*

## What is central-bank money?

Commercial-bank deposits and central-bank money are not the same thing. When a customer sees “$100” in a normal bank account, that balance is a liability of the commercial bank. Central-bank money is a liability of the central bank. In Australia, eligible financial institutions settle obligations in central-bank money through Exchange Settlement Accounts held at the RBA.

## What is a CBDC?

A central bank digital currency (CBDC) is a digital form of central-bank money. A retail CBDC would be designed for households and businesses to use directly. A wholesale CBDC - often discussed as tokenised central-bank reserves - would be restricted to financial institutions and other eligible wholesale participants and would be used for payment and settlement. The eAUD project was framed primarily around the wholesale/institutional problem, although its prototype also included a customer role to demonstrate wallet-level access and user flows.

## Payment versus settlement

A payment instruction says that value should move. Settlement is the point at which the financial obligation is finally discharged. In wholesale markets, settlement design matters because large institutions must know exactly when a transfer becomes final, what asset is being exchanged, which entity carries risk before finality and which legal/operational rules govern the process.

## What “tokenised” means in this report

Tokenisation represents rights or value as digital records on programmable infrastructure. The attraction is not that a database becomes “crypto”; it is that asset state, transaction rules and permissions can sometimes be coordinated on shared infrastructure. Project Acacia’s public findings in 2026 focused on how tokenised assets and different forms of digital money could improve the efficiency, functionality and resilience of wholesale markets, while also identifying legal, regulatory and scaling challenges.

### Not cryptocurrency

The prototype uses distributed-ledger technology, but the concept is fundamentally different from a public cryptocurrency. Participants are known, access is permissioned, identities are centrally managed, and the “currency” is a simulated research asset. There is no anonymous mining, speculative token issuance or public permissionless consensus.

# Australian context & problem framing

The problem is better understood as a tokenised-market coordination problem than as a claim that Australia's existing settlement system is slow or broken.

## Australia already has real-time gross settlement

The Reserve Bank Information and Transfer System (RITS) is Australia's high-value settlement system. Approved institutions settle payment obligations on a real-time gross settlement basis through Exchange Settlement Accounts. The RBA states that RITS settlement is final and irrevocable when the paying and receiving ESAs are simultaneously debited and credited. This matters because an early project presentation incorrectly described Australian RTGS as "T+2"; that framing is not used in this public case study.

## So what problem was worth exploring?

The more defensible research question is what happens when financial assets and market infrastructure become more programmable and tokenised. Traditional settlement systems are highly mature, but tokenised asset markets create new coordination questions: how digital assets interoperate with money, how institutions authenticate, how rights and roles are represented, how compliance checks integrate with new transaction flows, and how central-bank money remains available as a trusted settlement asset.

The RBA and DFCRC's Project Acacia tested 20 wholesale tokenised-asset-market use cases and multiple settlement methods, including existing ESA balances, a pilot wholesale CBDC, tokenised commercial-bank deposits and stablecoins. Its final findings identified potential benefits from tokenisation across issuance, servicing, trading and settlement while also emphasising that scaling requires further industry and regulatory work.

## The project's design challenge

Team P29 therefore treated the eAUD prototype as a controlled environment for asking four practical questions:

- How could different financial-system participants be represented with different rights and responsibilities?
- How could transaction logic be executed on a permissioned ledger without exposing every action to every user?
- How could KYC and suspicious-activity monitoring be integrated without putting sensitive documents directly on a shared ledger?
- How could a regulator-facing role observe risk information without being modelled as the party authorising commercial transactions?

04 / PROJECT BRIEF & SUCCESS CRITERIA

# Project brief & success criteria

The original brief combined research, architecture, policy and an operational roadmap. The software prototype grew out of that design work and client feedback.

## Original project direction

The Project A worklog records the brief as a comprehensive case study for designing and implementing an Australian eAUD concept. Required outputs included architectural design, policy analysis, an operational roadmap, evaluation of open-source blockchain approaches and integration of regulatory, technological and adoption considerations. This meant the project was never solely a coding exercise.

## How the scope evolved

In the first phase, the team researched CBDC models, stakeholder roles and policy considerations, built a stakeholder matrix, evaluated architecture choices and developed test cases. Client feedback asked for stronger storytelling, clearer test scenarios, better infographics and a stronger RBA/financial-institution focus. The team then moved from conceptual diagrams into a working Hyperledger Fabric network so that assumptions could be demonstrated rather than only described.

By Project B, the scope had become a full demonstrator: four network organisations, custom eAUD chaincode, five role-specific dashboards, KYC upload, AML risk scoring, enterprise identity, deployment orchestration and operational documentation. Later sprints added alerting, analytics, pagination/search and export capabilities.

## Practical success criteria

| Question | Evidence of success in the prototype |
|---|---|
| Separate institutions? | 4 Fabric organisations + 5 role-filtered application views. |
| Move simulated value? | Chaincode/API flows for wallet creation, funding and transfers. |
| Gate onboarding? | KYC upload and verification state before wallet creation. |
| Surface risk? | 7 monitoring signals, risk score, flags, dashboard and alert. |
| Centralise identity? | Keycloak group-based role mapping. |
| Reproduce the system? | Docker, realm export, manuals and GitHub documentation. |
| Demonstrate end-to-end? | Live transfer -> AML -> dashboard -> email -> search/pagination. |

05 / STAKEHOLDERS & OPERATING MODEL

# Stakeholders & operating model

The prototype maps financial-system actors into simplified technical roles so responsibilities can be discussed and tested.

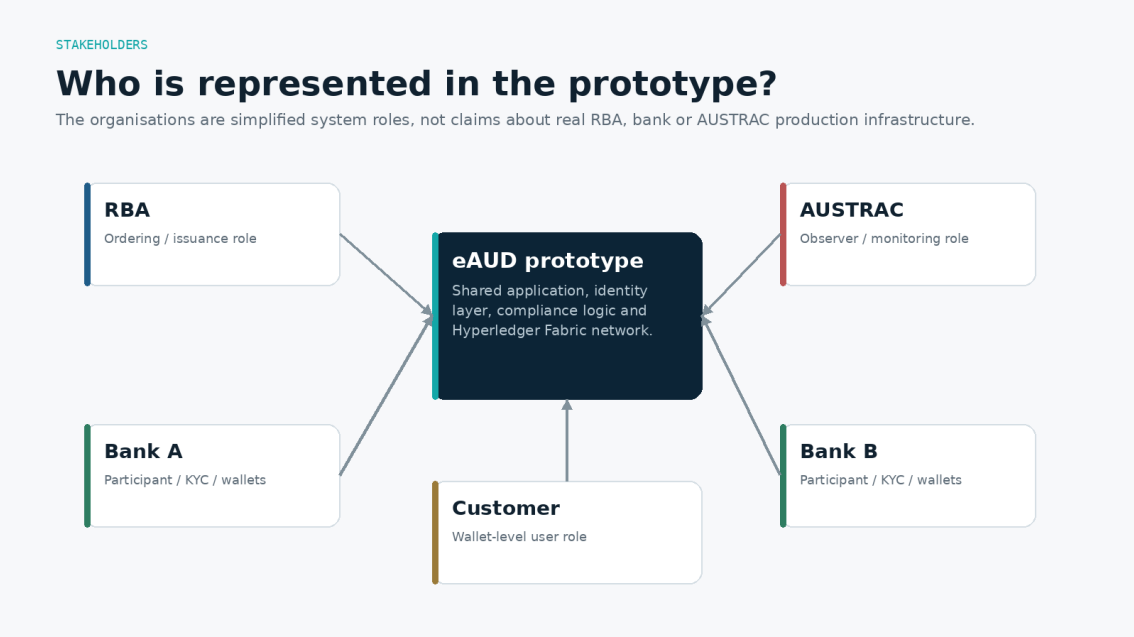



*Figure 2. Simplified stakeholder model used by the prototype.*

## RBA role

The RBA role has system-wide administrative visibility and simulated issuance/funding authority. At the Fabric layer, it is represented in the ordering architecture. This is a research abstraction, not a description of RBA production infrastructure.

## Commercial bank roles

Bank A and Bank B represent participating institutions. Their dashboards are scoped to their own wallets/KYC workflows and they are represented as endorsing peers, demonstrating organisational separation on a shared ledger.

## AUSTRAC role

AUSTRAC is represented as a regulator/observer. It sees monitoring, risk scores, suspicious flags, watchlist, filters, analytics and exports, but is not modelled as the user who creates wallets or initiates ordinary transfers.

## Customer role

The customer role provides a narrow wallet-level experience for balance/history and transfers. It demonstrates least-privilege access; it does not turn the broader project into a retail-CBDC design.

06 / SOLUTION ARCHITECTURE

# Solution architecture

A layered design separates identity, user experience, business/compliance logic and ledger state.

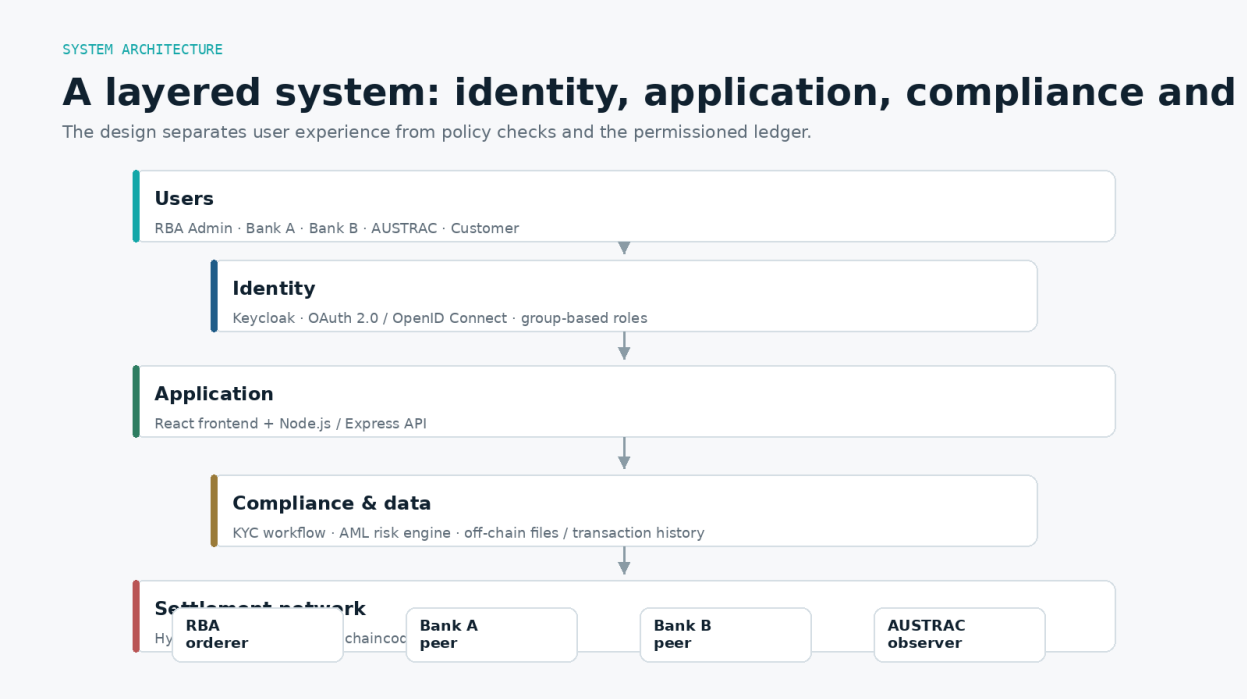



*Figure 3. High-level eAUD prototype architecture.*

## Layer 1 - Identity

Keycloak authenticates users and issues tokens containing group information. The frontend uses the role to choose the appropriate dashboard; the backend also validates role information before allowing restricted operations. This dual enforcement is important because hiding a button in a user interface is not security if the API still accepts the request.

## Layer 2 - Application

The React frontend provides role-specific screens for RBA administrators, bank administrators, AUSTRAC and customers. The Node.js/Express API acts as the coordination layer between user requests, KYC/AML logic, persistence and Hyperledger Fabric interactions.

## Layer 3 - Compliance and off-chain data

KYC documents are handled outside the blockchain. The prototype stored KYC files locally and used application-level state to gate wallet creation. Suspicious-activity logic also runs in the application layer because it needs transaction context, timestamps, prior history and watchlist state. Keeping sensitive documents off-chain reduces the risk of replicating personal information across ledger participants, although the prototype storage itself is not production-grade.

## Layer 4 - Permissioned ledger

Hyperledger Fabric provides organisational identity, peer nodes, an ordering service and chaincode execution. The project used a dedicated eAUD channel and CouchDB world state. Custom JavaScript chaincode exposed operations such as CreateWallet, AddFunds, TransferEAUD, QueryWallet and GetAllWallets.

### Why Hyperledger Fabric?

Fabric fit the project because it models known organisations and lets participants have distinct identities and endorsement roles. That is a closer conceptual fit for regulated institutions than a public permissionless blockchain. The choice does not prove Fabric is the right production architecture for an Australian CBDC; it made the governance assumptions visible and testable.

07 / A TRANSACTION FROM START TO FINISH

# A transaction from start to finish

Follow one transfer without needing to understand blockchain internals.

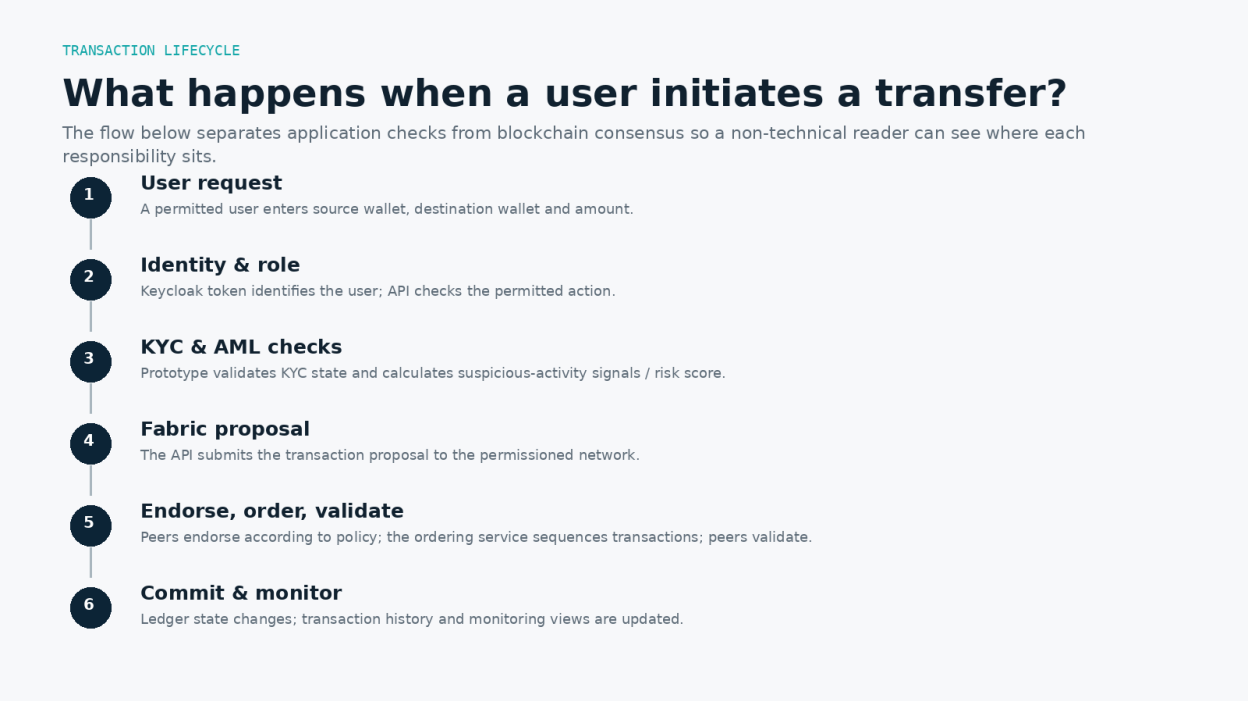



*Figure 4. Simplified transfer lifecycle in the prototype.*

## 1. A user requests a transfer

A logged-in user selects a source wallet, destination wallet and amount. The exact actions available depend on the authenticated role and the dashboard the user is permitted to access.

## 2. The API verifies identity and permissions

The browser sends an access token issued by Keycloak. The API reads the user's group/role information and determines whether the requested operation is allowed. Bank roles are restricted to their institutional scope; regulator and customer roles have different capabilities.

## 3. Application-level checks run

Before the ledger call, the application can validate KYC state and evaluate prototype AML signals. This is where contextual checks belong in the demonstrator because the API has access to transaction history, local watchlist data and application-specific policy.

## 4. The transfer is proposed to Fabric

Once application checks are satisfied, the backend uses the Fabric integration to submit the transaction. Peers execute/endorse according to the network policy, the ordering service places transactions in a consistent sequence and peers validate/commit the resulting block.

## 5. Operational views are updated

After the transaction is processed, the application records history and displays the result. AUSTRAC-facing monitoring can show the amount, status, risk score and triggered flags. If the configured risk threshold is exceeded, the notification subsystem can send an email alert.

### Settlement versus monitoring

The prototype risk engine is a demonstration of transaction monitoring. It is not a legal "approval engine" for Australian payments and it does not automatically create a Suspicious Matter Report (SMR). In production, reporting decisions, timeframes, data quality, escalation and audit obligations would require a much more rigorous control framework.

08 / PRODUCT WALKTHROUGH

# Product walkthrough

The interfaces were designed around role boundaries: issuance/administration, institutional onboarding, regulatory monitoring and wallet-level access.

## RBA administrative view

The RBA-facing dashboard provides the broadest view of the system. It supports system-level wallet operations, simulated eAUD funding/minting, transfers and transaction history. The final version also applied the KYC requirement to RBA-created wallets after client feedback identified an inconsistent exemption.

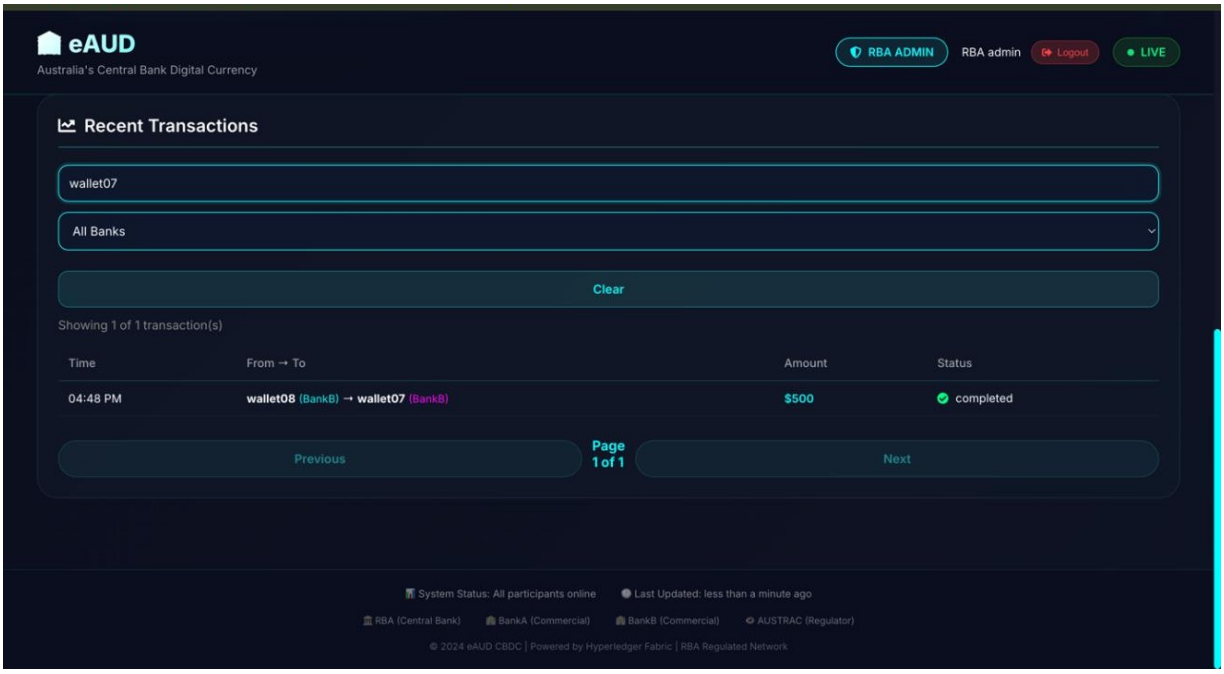



*Figure 5. RBA transaction history view with filters, search and pagination. Pagination/search was integrated from team member Lasith Perera's Sprint 4 work.*

## Bank onboarding and KYC

Bank administrators are scoped to their institution. The KYC flow requires an approved document type and captures fields such as Legal Name and Purpose of Account before wallet creation. The UI displays KYC-verified status on wallet records. This is a prototype onboarding workflow rather than a complete customer due-diligence system; it demonstrates where onboarding state influences wallet access.

## AUSTRAC monitoring

The AUSTRAC dashboard is the clearest expression of the project's compliance-first design. Rather than hiding monitoring in logs, suspicious flags and risk scores are visible as first-class operational data. In the final demo, a $100,000 test transfer triggered the rules, produced a high risk score and appeared as a flagged transaction.

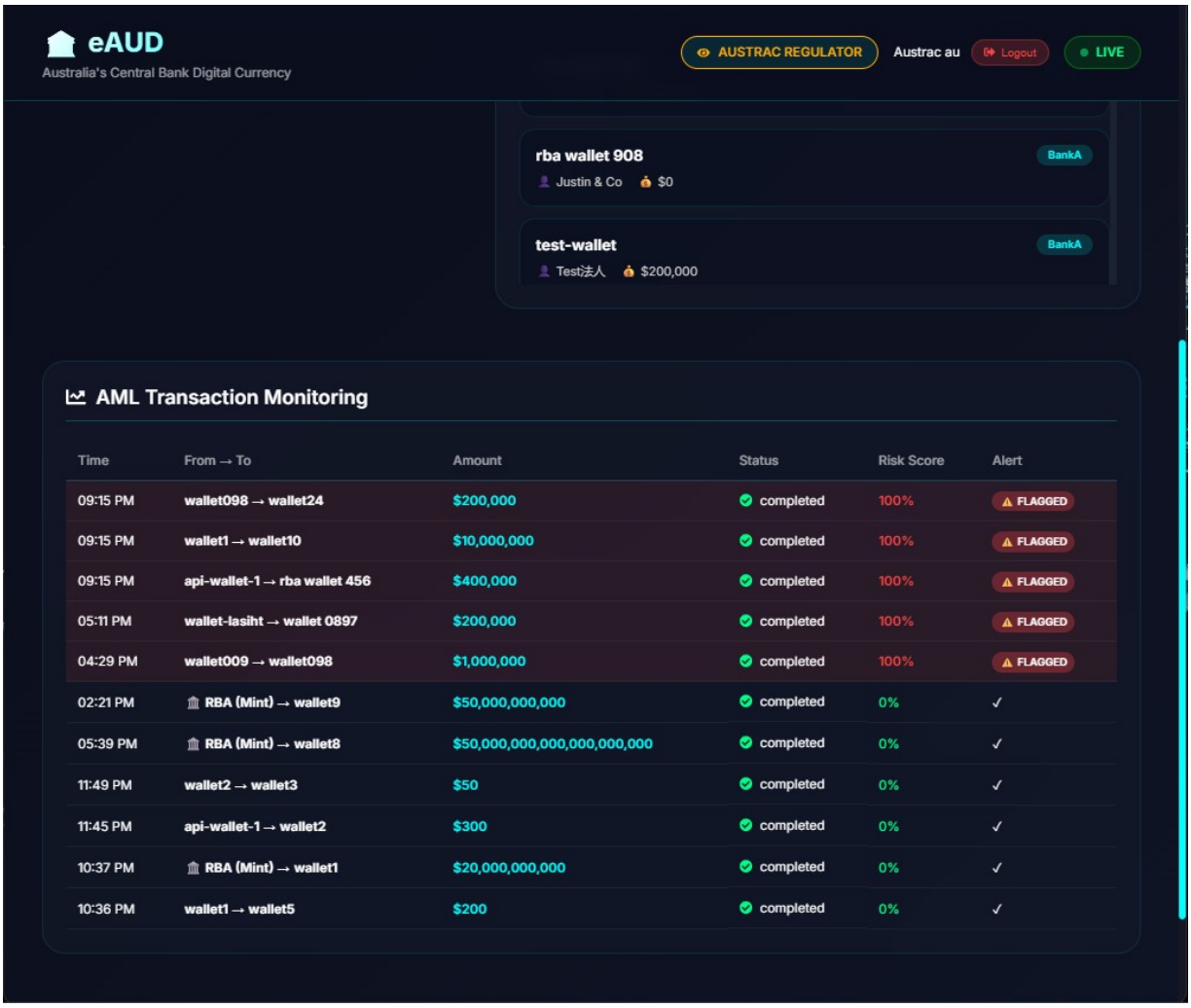



*Figure 6. AUSTRAC AML Transaction Monitoring dashboard after the integration fix. The red rows show flagged prototype transactions and risk scores.*

## Portfolio-level analytics

The analytics extension moved beyond individual transactions. Team member Ashikur Rahman implemented transaction volume over time, transactions by bank and high-risk/low-risk visualisations using Chart.js. These views were designed to respond to active filters and help the regulator role identify patterns across the transaction set.

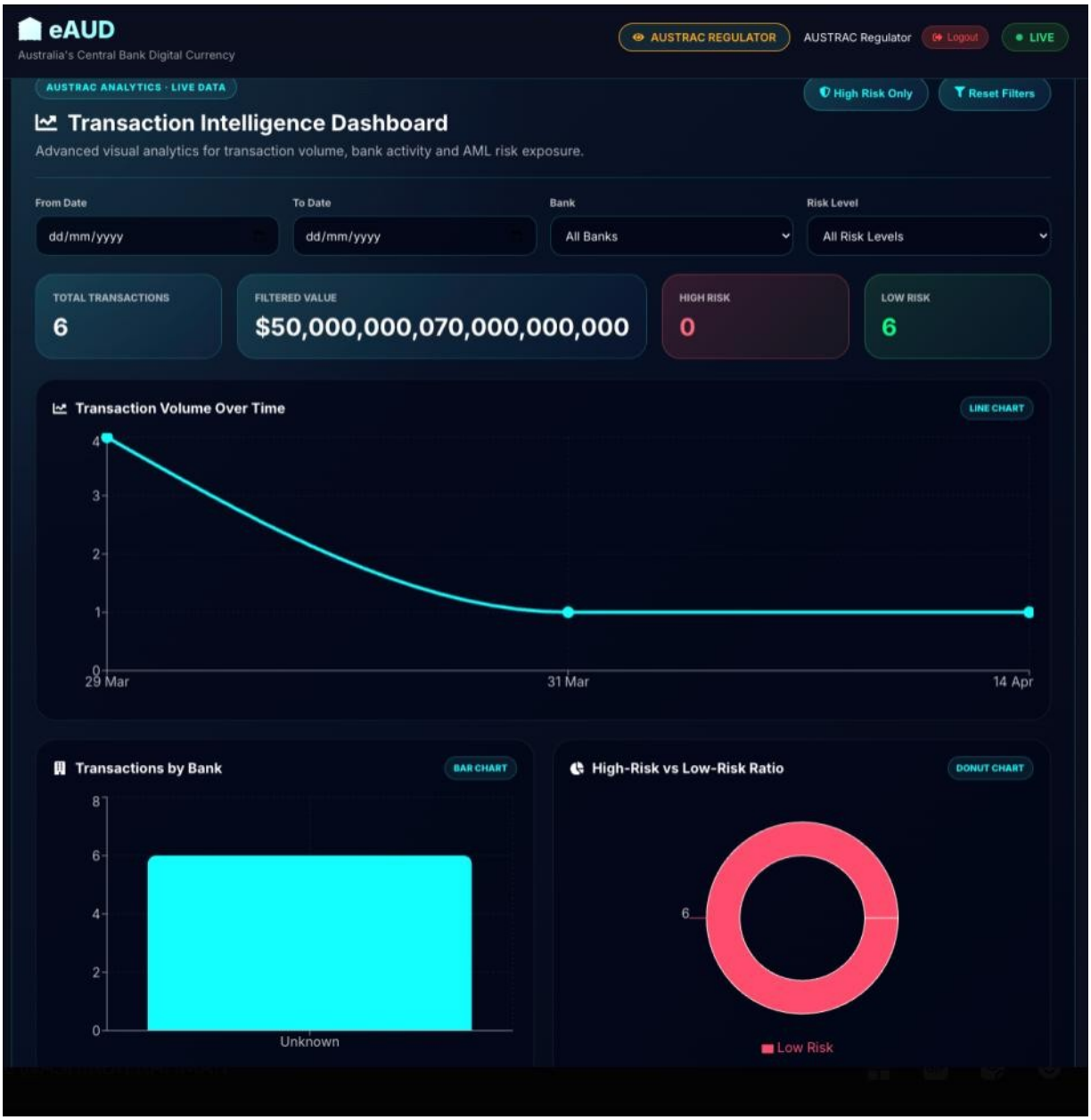
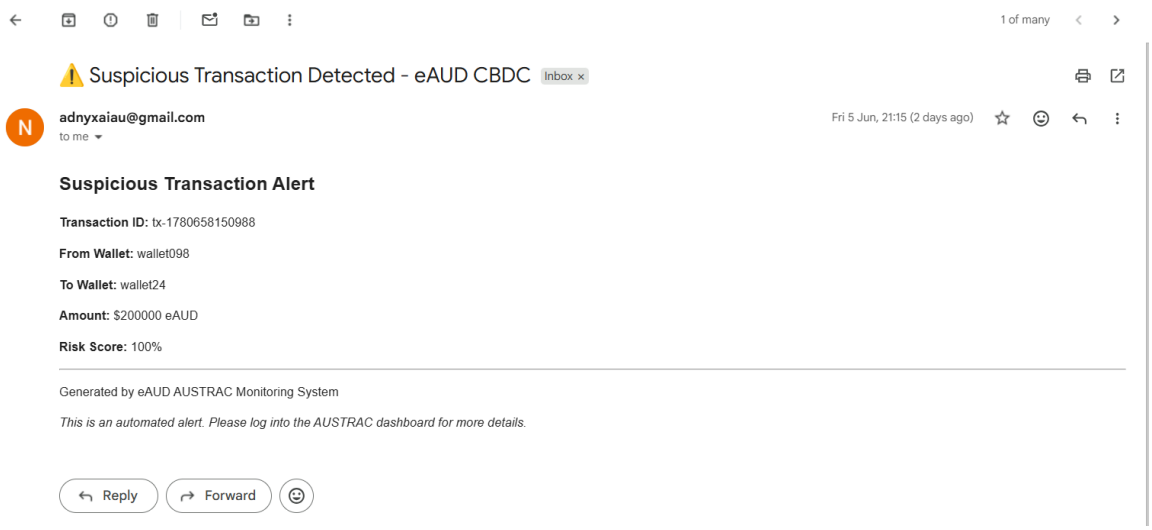



*Figure 7. AUSTRAC transaction-intelligence dashboard with volume, bank and risk-distribution charts.*

## Alerts and reporting

The monitoring workflow was extended with automated email alerts and export functions. Farzana Moietry implemented the initial suspicious-transaction email service using Nodemailer. Luvish Rajnath implemented PDF export for filtered AML transactions, while Al Hamid Arath implemented CSV export. These features turned the dashboard into a more complete reporting workflow rather than a read-only screen.



1 of many

⚠️ Suspicious Transaction Detected - eAUD CBDC Inbox ×

N **adnyxaiau@gmail.com**
to me

Fri 5 Jun, 21:15 (2 days ago)

**Suspicious Transaction Alert**

**Transaction ID:** tx-1780658150988

**From Wallet:** wallet098

**To Wallet:** wallet24

**Amount:** $200000 eAUD

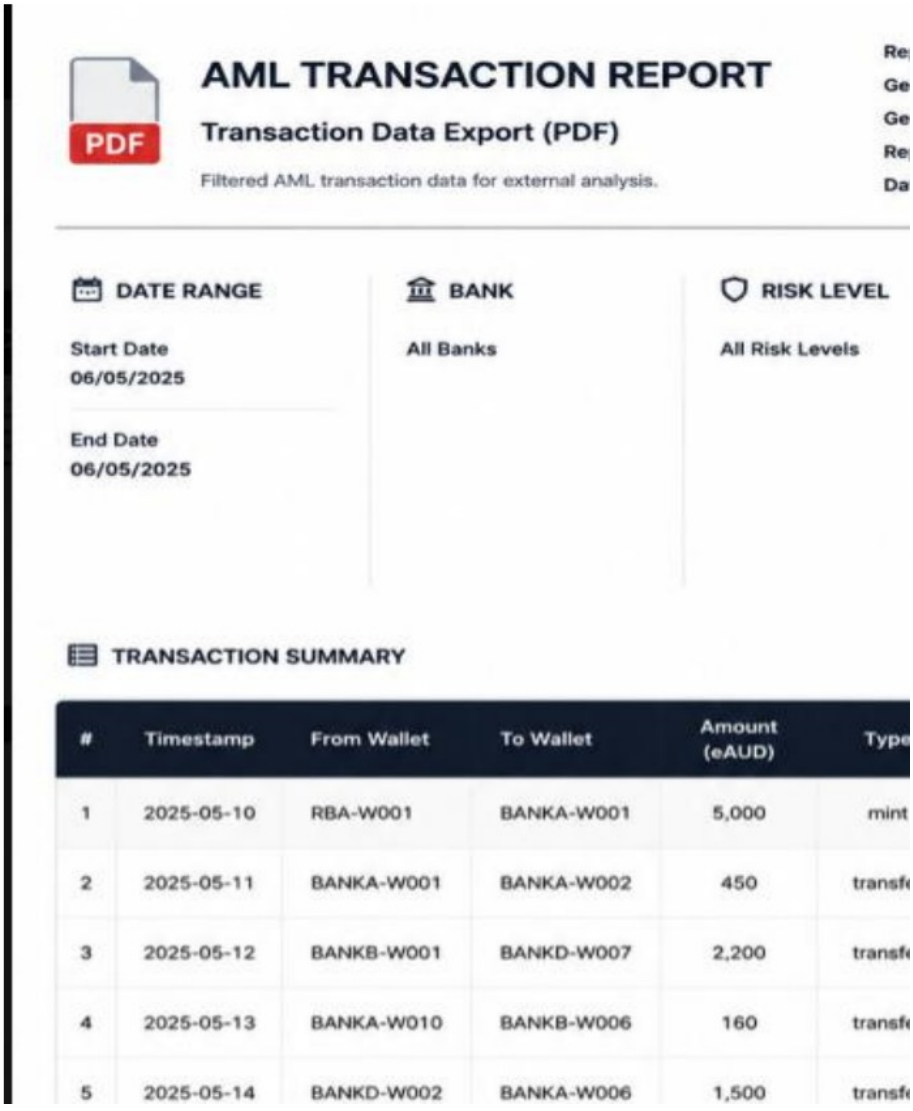
**Risk Score:** 100%

Generated by eAUD AUSTRAC Monitoring System

*This is an automated alert. Please log into the AUSTRAC dashboard for more details.*

Reply Forward

*Figure 8. Example automatic email alert generated when a prototype transaction exceeded the configured risk-score threshold.*



PDF

# AML TRANSACTION REPORT

## Transaction Data Export (PDF)

Filtered AML transaction data for external analysis.

**DATE RANGE**

Start Date
06/05/2025

End Date
06/05/2025

**BANK**

All Banks

**RISK LEVEL**

All Risk Levels

**TRANSACTION SUMMARY**

| # | Timestamp | From Wallet | To Wallet | Amount (eAUD) | Type |
|---|---|---|---|---|---|
| 1 | 2025-05-10 | RBA-W001 | BANKA-W001 | 5,000 | mint |
| 2 | 2025-05-11 | BANKA-W001 | BANKA-W002 | 450 | transf |
| 3 | 2025-05-12 | BANKB-W001 | BANKD-W007 | 2,200 | transf |
| 4 | 2025-05-13 | BANKA-W010 | BANKB-W006 | 160 | transf |
| 5 | 2025-05-14 | BANKD-W002 | BANKA-W006 | 1,500 | transf |

*Figure 9. Example AML transaction report generated by the project's PDF export feature.*

09 / AML/CTF MONITORING PROTOTYPE

# AML/CTF monitoring prototype

A demonstration of how suspicious-activity signals can be combined with transaction processing - with important boundaries around what the model means.

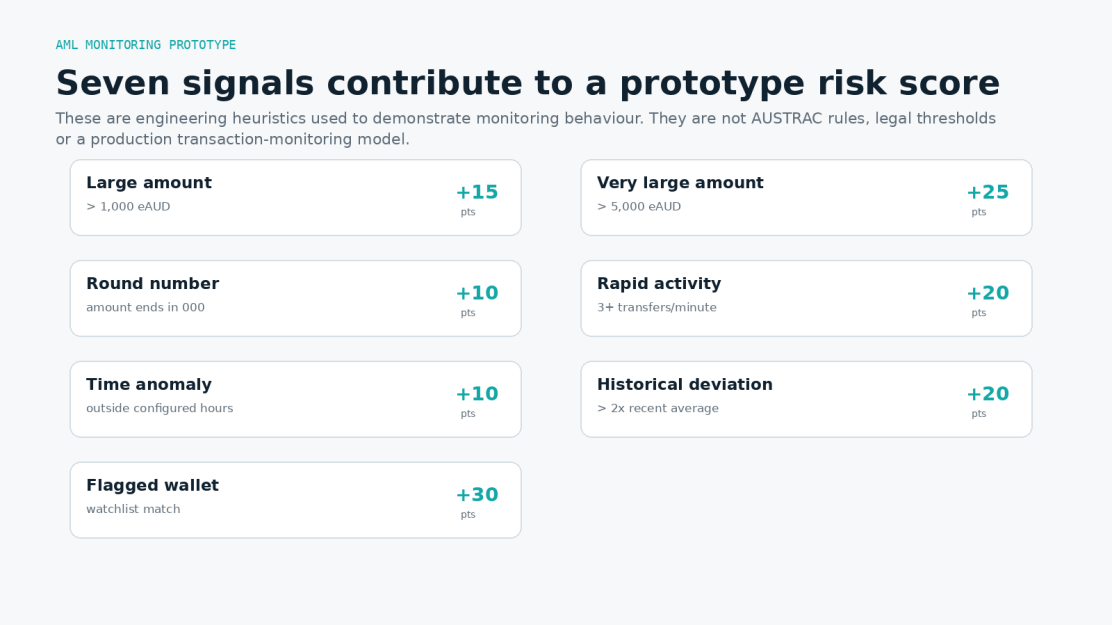



*Figure 10. Seven prototype AML monitoring signals and their configured risk-score increments.*

## Why the engine exists

The goal was to demonstrate that transaction monitoring can be integrated into the same user journey as digital settlement. Each transfer is evaluated against a set of heuristics, triggered flags are collected and a risk score is calculated. A threshold can then drive an operational alert and highlight the transaction for the regulator-facing dashboard.

## The seven signals

The prototype uses amount thresholds, round-number patterns, rapid transaction frequency, time-of-day behaviour, deviation from a wallet's recent average and a flagged-account watchlist. These signals are familiar examples of transaction-monitoring logic, but the exact values were chosen for demonstration. They should not be interpreted as AUSTRAC-prescribed thresholds or as a validated statistical model.

## A useful design distinction: alerting is not reporting

AUSTRAC's real Suspicious Matter Report regime is based on a reporting entity forming a relevant suspicion, not on a single static numeric threshold. AUSTRAC's current guidance requires an SMR within 24 hours for terrorism-financing suspicions and within three business days for other suspicions. The eAUD prototype does not submit SMRs, make legal determinations or replace human investigation. Its email alert is simply an internal operational notification.

## Calibration lesson

The final client feedback identified a major modelling limitation: the “large” and “very large” thresholds were too low for a wholesale institutional context. That feedback is important because a monitoring model that flags normal institutional behaviour creates noise and undermines the usefulness of alerts. A production approach would require scenario design based on actual participant types, product behaviour, historical distributions, typologies, risk appetite, explainability and ongoing tuning.

# Identity, KYC & security design

Financial systems need more than a login page. Identity, role boundaries and sensitive data handling must be designed as system controls.

## From hard-coded JWT prototype to Keycloak

The first implementation used a simple JWT-based authentication approach suitable for fast prototyping. In Sprint 3, the project migrated to Keycloak using OAuth 2.0/OpenID Connect concepts and group-based roles. The Keycloak realm defined the five application roles and allowed the team to test them through one identity provider.

1. Signing with rba in one session using keycloak

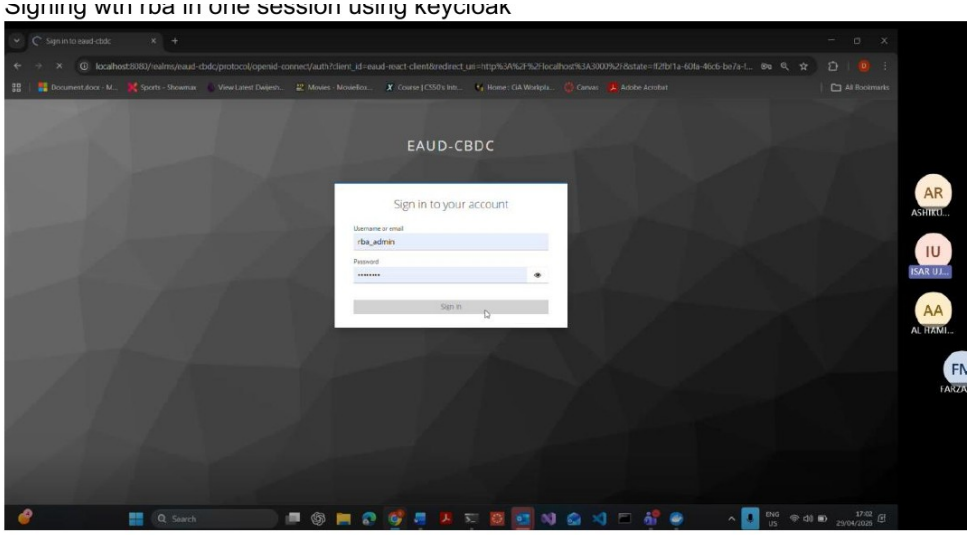



*Figure 11. Keycloak sign-in used by the prototype. The screenshot is from the Project B worklog evidence.*

## The group-mapping bug

Authentication succeeded before authorisation did. Keycloak returned group paths with a leading slash, such as /rba_admin, while the application expected rba_admin. The team diagnosed the issue by inspecting token claims and normalised the group names before role comparison. It is a small code change with a larger lesson: identity integration fails at the boundaries between systems, and a valid token does not automatically mean application permissions are correct.

## KYC data and privacy

The design deliberately kept KYC documents off-chain. That is directionally sensible because a shared ledger is a poor place to replicate sensitive identity documents. The regulatory alignment work also considered the Privacy Act and APP obligations. However, the prototype used local file storage and therefore does not claim production privacy or security compliance.

## Security expectations in a real financial institution

APRA CPS 234 requires regulated entities to maintain information-security capability commensurate with threats, protect information assets, test controls and manage/report material security incidents. The ACSC Essential Eight similarly provides a baseline set of mitigation strategies such as

application control, patching, multi-factor authentication, administrative privilege restrictions and backups. In the project, these sources informed design direction; the prototype itself did not undergo formal security accreditation or control assurance.

# Engineering implementation

The prototype is a multi-service application rather than a standalone blockchain script.

## Hyperledger Fabric and chaincode

The Fabric network evolved from a basic two-peer experiment in Project A into a four-organisation model in Project B. Custom JavaScript chaincode implemented wallet state and core eAUD operations. The network used CouchDB for world state and an eAUD channel for shared ledger operations.

## Node.js / Express API

The API coordinates authentication, wallet operations, KYC upload, transfers, suspicious-activity detection, transaction history, flagged accounts and export/notification integration. This layer became the practical boundary between user-facing workflows and ledger execution.

## React application

The React frontend uses role-based conditional rendering to expose different dashboards. The RBA role sees system-wide administrative functions, bank roles see institution-scoped wallet/KYC functions, AUSTRAC sees monitoring and reporting, and the customer role sees wallet-level information and transfers.

## Persistence and prototype trade-offs

Some application state was persisted in JSON files rather than a production database. This fixed an early issue where transaction history disappeared when the API restarted and made demos reliable, but it is explicitly a prototype compromise. A production system would need transactional persistence, access controls, encryption, backup/restore, data-retention policy and operational monitoring.

## Dockerisation

Environment consistency became a delivery problem because the project combined Windows/WSL, Fabric containers, Keycloak, Node and React. Dockerfiles were created for the API and frontend and Docker Compose was used to orchestrate the application services. The final handover also included the Keycloak realm export and installation documentation so another developer could recreate the identity configuration.

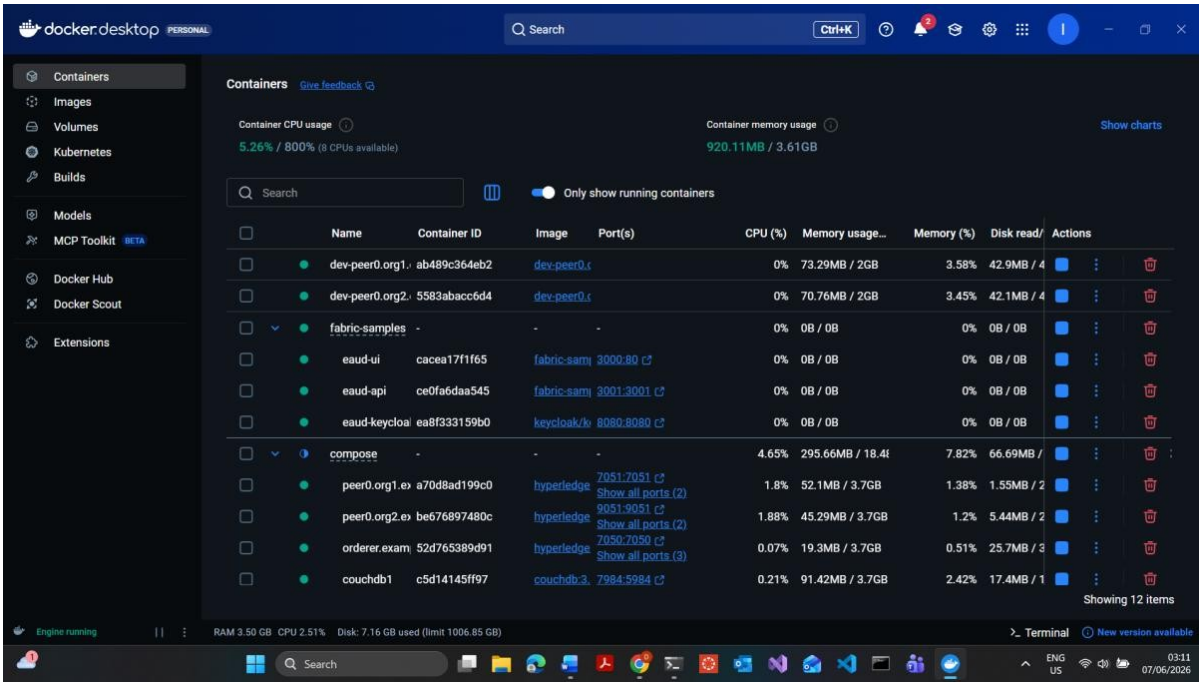



*Figure 12. Docker Desktop showing project-related containers and supporting services running during the final sprint.*

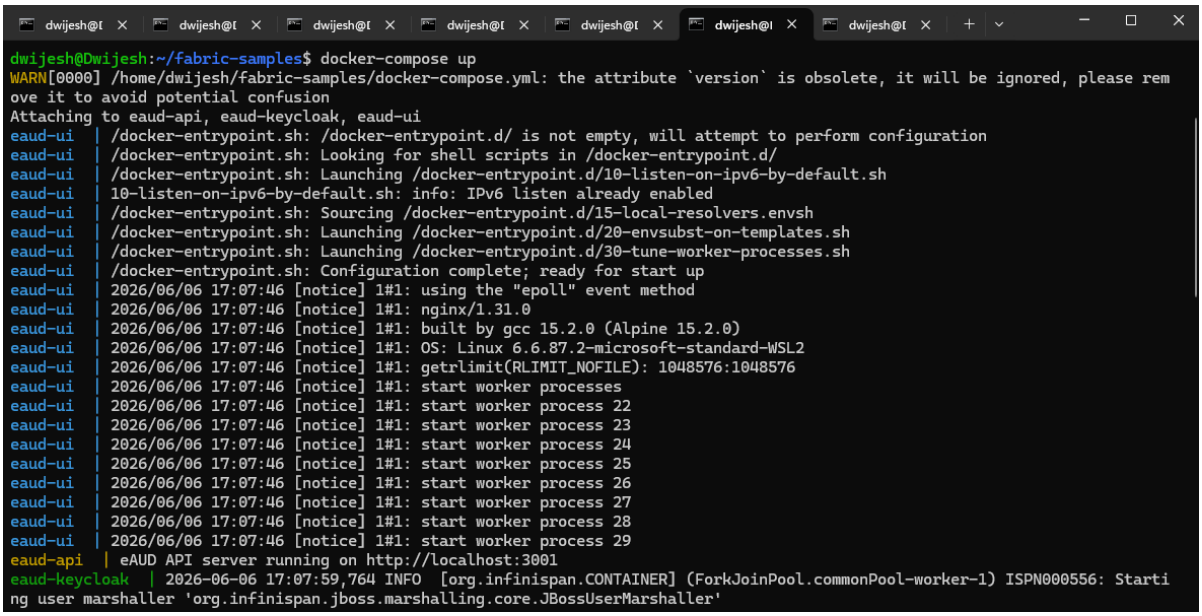



*Figure 13. Terminal evidence from the containerised application environment.*

12 / DELIVERY PROCESS & PROJECT EVOLUTION

# Delivery process & project evolution

Two academic phases, repeated demonstrations and client feedback transformed the project from research framing into an integrated prototype.

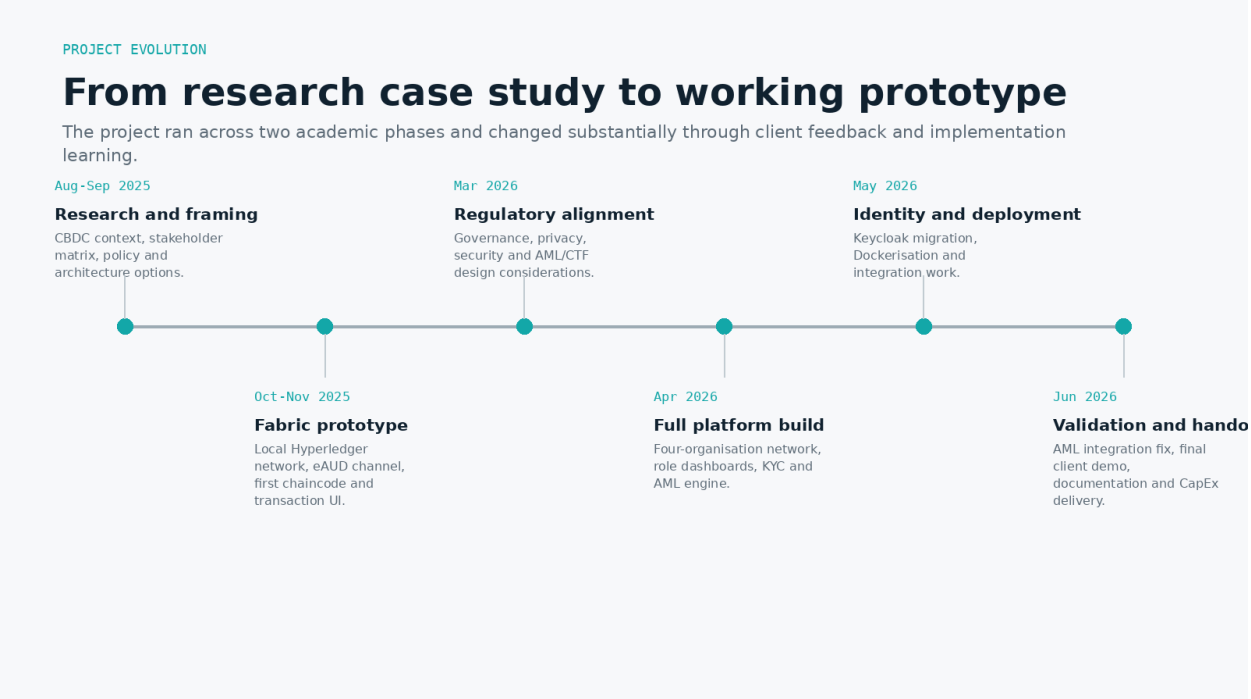



*Figure 14. Project evolution across 2025-2026.*

## Project A - research and first prototype

The first phase concentrated on the business/technology domain, stakeholder mapping, architecture and regulatory research. By Weeks 8-11, the worklog shows the transition into Hyperledger Fabric setup, a simple network, channel creation, basic chaincode deployment and transactions between the initial RBA/Bank A participants. A simple UI was then added to visualise transactions.

## Project B - regulatory alignment and platform build

The second phase began with a dedicated regulatory-alignment stream covering governance, data/privacy, cryptography/security, AML/CTF and cross-border considerations. The platform build then expanded rapidly: four organisations, custom chaincode, API, five dashboards, KYC and AML logic. Later sprints focused on Keycloak, Dockerisation, integration, reporting and handover.

## Client demonstrations as requirements discovery

The project used frequent demonstrations rather than waiting until the end. Feedback directly changed the product: KYC fields were refined, KYC was enforced more consistently, Keycloak was adopted, deployment reproducibility became a requirement and AML thresholds were challenged as too low for wholesale use. This is one of the project's strongest process outcomes: demonstrations exposed mismatches between implementation assumptions and stakeholder expectations while there was still time to respond.


*Figure 15. CapEx poster produced for the final exhibition. It summarised the project using the Problem / Process / Solution structure.*

13 / TESTING, DEFECTS & LESSONS

# Testing, defects & lessons

The project's most valuable engineering lessons came from failures that crossed service boundaries.

## Case 1 - the AML engine existed but was not in the transaction path

The detectSuspicious() function had been implemented, but the /api/transfer endpoint did not call it. As a result, the compliance core could appear complete in code while remaining inactive in the actual user journey. The defect survived multiple demonstrations because testing focused on components instead of tracing the complete transaction path. End-to-end testing eventually exposed the missing integration call; once connected, the seven rules produced risk scores and suspicious flags on the AUSTRAC dashboard.

**Lesson**

A correct function is not the same thing as a correct system. For compliance-sensitive behaviour, tests must prove that the control is reached in the real transaction path, that its output is stored, that downstream actions occur, and that failure conditions behave safely.

## Case 2 - Keycloak authentication worked, role mapping did not

Users could authenticate but still receive a broken or undefined role because the token's groups claim did not match the application's expected string format. Inspecting the token payload revealed the leading slash. The fix was straightforward; diagnosing the boundary between identity provider, token and application logic was the difficult part.

## Case 3 - chaincode and endorsement configuration

During Fabric development, custom chaincode containers and endorsement policies caused silent or confusing failures. The team reverted to known-working examples and reintroduced custom code incrementally to isolate the problem. The development endorsement policy was changed to a form that allowed progress while the network design was still evolving.

## Case 4 - reproducible startup

Multi-service startup repeatedly failed when dependent services were not ready. Dockerisation forced the team to make service dependencies explicit and reduced environment drift between developers. In a production system, this would need to go further with robust health checks, observability, secrets management and automated deployment pipelines.

## Testing maturity gained

By the final sprint, testing covered all five role flows, Keycloak login/permissions, KYC state, transfers, risk scoring and containerised operation. The final live demonstration intentionally exercised a complete high-risk transaction: transfer -> AML detection -> AUSTRAC display -> email notification -> transaction search/pagination.

# Team contributions & authorship

This is a Team P29 project. The case study is authored by Isar Ujoodah, but product features and final delivery were shared across six team members.

## Team P29

The final project team comprised Isar Ujoodah, Luvish Rajnath, Lasith Perera, Farzana Moietry, Ashikur Rahman and Al Hamid Arath. The group contributed across research, reports, testing, client demonstrations, diagrams/poster work and feature implementation. The table below records the explicit Sprint 4 feature ownership documented in the team report.

| Team member | Sprint 4 documented role / feature ownership |
|---|---|
| Isar Ujoodah | Scrum Master / lead developer; Dockerisation, AML integration fix, KYC fix, final demo, GitHub coordination, CapEx poster, sprint-plan/review and testing/integration. |
| Luvish Rajnath | PDF export for AML reports; sprint/project retrospective support and testing. |
| Lasith Perera | Transaction-table pagination and wallet-ID search; final delivery/feedback documentation and testing. |
| Farzana Moietry | Suspicious-transaction email notification system; project review and testing. |
| Ashikur Rahman | AUSTRAC transaction analytics charts; project retrospective and testing. |
| Al Hamid Arath | CSV export for AML reports; project review and testing. |

## Author’s contribution

I served as Scrum Master and lead developer and was responsible for a large share of the core integration work across both phases. My documented work includes the Fabric network setup and extension, custom eAUD chaincode, major parts of the Node/Express API and React dashboards, KYC/AML implementation, the Keycloak migration, Dockerisation, debugging of the AML integration failure, client coordination and final handover documentation.

That contribution is significant, but it should not erase the team nature of the final product. The reporting, alerting, analytics and export capabilities were strengthened by other team members and were integrated into the final system. This case study therefore treats “the project” as Team P29’s outcome and identifies personal ownership only where useful for transparency.

15 / EVALUATION, LIMITATIONS & FUTURE WORK

# Evaluation, limitations & future work

The prototype is useful because it reveals design trade-offs - not because it proves a production CBDC should look exactly like this.

## What the prototype demonstrated well

- Institutional roles can be made explicit at both the application and permissioned-ledger layers.
- KYC state and transaction-monitoring logic can be connected to the same transaction journey as ledger operations.
- A regulator-facing monitoring experience can be separated from transaction-authorising roles.
- Enterprise identity changes the quality of a prototype by making authentication and authorisation testable across multiple user types.
- Reproducible deployment and documentation are essential for handover of a multi-service prototype.

## Important limitations

| Area | Limitation / implication |
|---|---|
| Financial-market realism | The prototype simplifies real wholesale-market structures, participants, legal arrangements and settlement assets. |
| RBA representation | The ordering/issuance role is a research abstraction, not a description of actual RBA architecture or policy. |
| AML/CTF | Seven heuristic rules and fixed risk weights are demonstration logic, not validated typologies, thresholds or AUSTRAC reporting logic. |
| KYC | Document upload/verification is a simplified workflow and does not implement complete customer due diligence, identity verification or record-keeping obligations. |
| Security | No formal penetration test, threat model, APRA assurance, Essential Eight maturity assessment or production secrets/key management. |
| Data | Local/file-based persistence is suitable for a demo, not a resilient financial-system data architecture. |
| Blockchain governance | Endorsement, membership, key custody, certificate lifecycle, upgrade policy and incident governance require much deeper design. |
| Scale | The project did not benchmark throughput, latency, high availability, disaster recovery or institution-scale monitoring |

| Area | Limitation / implication |
|---|---|
| | loads. |
| Interoperability | No integration with RITS, ISO 20022 payment rails, commercial-bank core systems or tokenised asset platforms. |

## What should come next

A serious next phase would begin by tightening the research question. Rather than “build an eAUD”, the next prototype should pick a specific wholesale use case - for example settlement of a tokenised debt instrument or interbank transfer between controlled participants - and define the legal asset, participants, settlement finality, data classification and failure states before implementation.

- Replace local JSON/file storage with a secure database and encrypted object storage, with audit logs and retention controls.
- Add automated unit, integration and end-to-end tests that prove identity, KYC, monitoring and ledger calls are connected.
- Adopt MFA, secrets management, certificate/key lifecycle controls and a documented threat model.
- Replace static AML weights with a calibrated scenario framework and investigation workflow, clearly separated from statutory reporting.
- Instrument the platform with observability, metrics and immutable audit evidence.
- Design interoperability around real market messages and settlement interfaces rather than treating the blockchain as an isolated system.

The broader Australian context also continues to evolve. Project Acacia’s 2026 findings moved the conversation from a generic “CBDC yes/no” question toward the practical design of tokenised wholesale markets, interoperability and settlement infrastructure. That makes the project’s strongest legacy its architecture questions: identity, governance, compliance placement, data boundaries and end-to-end controls.

APPENDIX A

# Glossary

| Term | Plain-English meaning |
|---|---|
| ADI | Authorised deposit-taking institution, such as a bank, regulated by APRA. |
| AML/CTF | Anti-money laundering and counter-terrorism financing. |
| AUSTRAC | Australian Transaction Reports and Analysis Centre; Australia's financial intelligence unit and AML/CTF regulator. |
| CBDC | Central bank digital currency - digital central-bank money. |
| Central-bank money | A liability of the central bank, used as the trusted settlement asset at the core of the monetary system. |
| Chaincode | Hyperledger Fabric term for smart-contract/business logic executed by peers. |
| CouchDB | Database used by Fabric for world-state storage in this prototype. |
| Endorsement | Fabric process in which required peers execute/sign a transaction proposal before ordering and validation. |
| ESA | Exchange Settlement Account held at the RBA by eligible institutions. |
| KYC | Know Your Customer - processes used to identify/verify customers and understand the relationship/risk. |
| OAuth 2.0 / OIDC | Standards used for delegated authorisation and identity/login flows; Keycloak implemented the project's identity layer. |
| Permissioned ledger | A ledger where participants are known and access is controlled. |
| RITS | Reserve Bank Information and Transfer System, Australia's high-value settlement system. |
| RTGS | Real-time gross settlement - individual payments settle in real time and with finality. |
| SMR | Suspicious Matter Report submitted to AUSTRAC by reporting entities when statutory suspicion/reporting criteria are met. |
| Tokenisation | Representing an asset, claim or form of money as a programmable digital record on shared infrastructure. |
| Wholesale CBDC | Central-bank digital money intended for a limited range of wholesale financial-market participants. |

APPENDIX B

# References & project evidence

The case study distinguishes project evidence from public context. Project artefacts support what Team P29 built; official Australian sources support public financial-system and regulatory context.

## Project evidence

P1. Team P29, Project A Individual Worklog, 2025. Covers initial brief, stakeholder research, Hyperledger prototype and early demos.

P2. Team P29 / Isar Ujoodah, Project B Individual Worklog, 2026. Covers regulatory alignment, full platform build, Keycloak, Dockerisation, integration testing and final demo.

P3. Team P29, Sprint Four Report, 8 June 2026. Documents final feature ownership, AML integration fix, Dockerisation, exports, analytics, email alerts and final handover.

P4. Team P29, Regulatory Alignment Report, 2026. Covers governance, data/privacy, security, AML/CTF and cross-border design considerations.

P5. Team P29, System Installation Manual, 2026. Documents application services, prerequisites, setup, role verification and repository structure.

P6. Team P29, final presentation and Week 12 demo artefacts, June 2026.

## Public context and regulatory sources

[1] Reserve Bank of Australia. About RITS. https://www.rba.gov.au/payments-and-infrastructure/rits/about.html

[2] Reserve Bank of Australia. RBA and DFCRC Release Findings From Project Acacia, 18 May 2026. https://www.rba.gov.au/media-releases/2026/mr-26-13.html

[3] Reserve Bank of Australia. Project Acacia - In Brief. https://www.rba.gov.au/payments-and-infrastructure/tokenised-money/project-acacia/

[4] Reserve Bank of Australia. About Tokenised Money. https://www.rba.gov.au/payments-and-infrastructure/tokenised-money/about.html

[5] AUSTRAC. Suspicious matter reports - submission deadlines. https://www.austrac.gov.au/industry-and-business/obligations-and-guidance/your-amlctf-program/reporting-us/suspicious-matter-reports

[6] APRA. CPS 234 Information Security. https://www.apra.gov.au/standards/cps-234

[7] Australian Signals Directorate / Cyber.gov.au. Essential Eight maturity model. https://www.cyber.gov.au/business-government/asds-cyber-security-frameworks/essential-eight/essential-eight-maturity-model

## Repository

Team P29 public GitHub repository: github.com/Dwijesh07/P29-Designing-Australia-s-Central-Bank-Digital-Currency

CLOSING NOTE

# A case study about the system, not a claim about the future of money

The eAUD prototype is most useful as a record of design decisions, integration problems and the questions that appear when finance, compliance and software architecture meet.

The project began as a broad Australian CBDC case study and became a working multi-service prototype. That evolution made abstract issues tangible: whether KYC belongs on a ledger, how regulatory visibility differs from transaction authority, where monitoring should run, how role-based access should be enforced, and how quickly a system can appear correct when a critical function is not actually connected to the user journey.

For Team P29, the outcome was not a blueprint for the Reserve Bank. It was a practical engineering exercise in turning policy and architecture questions into software that could be demonstrated, challenged, debugged and handed over.

**Project by Team P29**

Isar Ujoodah · Luvish Rajnath · Lasith Perera · Farzana Moietry · Ashikur Rahman · Al Hamid Arath

Case-study author: Isar Ujoodah · Swinburne University of Technology · 2025-2026